Actividad 2 – Informe de migración de datos

Introducción.

El objetivo de esta actividad fue migrar los datos de la base de datos Sakila desde MySQL hacia SQL Server, adaptando tanto la sintaxis como la estructura de los scripts para que fueran compatibles con el nuevo motor de base de datos. Además de migrar los registros, fue necesario conservar la integridad de la información, respetar las relaciones entre las tablas y mantener la consistencia del modelo de datos traducido al español.

Método utilizado para la carga de datos.

La migración se realizó utilizando los archivos de inserción (INSERT) proporcionados por la base de datos Sakila. Cada uno de estos scripts fue adaptado manualmente para que pudiera ejecutarse correctamente en SQL Server.
Durante este proceso fue necesario eliminar instrucciones exclusivas de MySQL, como SET AUTOCOMMIT y COMMIT. También se tradujeron los nombres de las tablas y de las columnas al español para que coincidieran con el modelo de datos desarrollado durante la primera parte del trabajo.
En aquellas tablas que poseen una columna IDENTITY, se utilizó la instrucción SET IDENTITY_INSERT ON, permitiendo conservar los identificadores originales de los registros durante la migración.

Manejo de grandes volúmenes de datos.

Una de las principales dificultades que encontré fue que SQL Server permite insertar un máximo de 1000 filas por sentencia INSERT ... VALUES.
Por ese motivo, las tablas con una gran cantidad de registros, como pelicula_actor, inventario, alquiler y pago, fueron divididas en bloques de hasta 1000 registros, separados por una instrucción GO.
Este fue uno de mis principales aprendizajes obtenidos durante la actividad, ya que inicialmente intenté ejecutar los archivos completos y SQL Server rechazaba la carga.

Uso de transacciones y manejo de errores.

Con el objetivo de realizar una migración segura, en los scripts se incorporó la instrucción:
SET XACT_ABORT ON;
Esta configuración indica a SQL Server que, si ocurre cualquier error durante una transacción, todos los cambios realizados hasta ese momento deben deshacerse automáticamente.
Además, los procesos de carga se ejecutaron utilizando bloques:
BEGIN TRY
...
END TRY

BEGIN CATCH
...
END CATCH
Esta estructura permitió capturar cualquier error producido durante la ejecución del script y realizar un ROLLBACK, evitando que la base de datos quedara parcialmente cargada o con información inconsistente.
El de estos fue incorporado luego de investigar sobre buenas prácticas para la migración en SQL Server y consultar una herramienta de asistencia basada en inteligencia artificial.

Adaptación de los triggers.

Durante el análisis se observó que varios de ellos únicamente tenían la función de completar automáticamente la fecha utilizando la función NOW() cuando se realizaba un nuevo INSERT.
Como los archivos de migración ya incluían esos valores de fecha no se migraron.
En el caso de la tabla pago, la funcionalidad del trigger fue reemplazada por un DEFAULT GETDATE(), solución equivalente y recomendada en SQL Server para asignar automáticamente la fecha actual cuando no se especifica un valor.

Dificultades encontradas.

Una de ellas fue la existencia de una dependencia circular entre las tablas empleado y tienda, ya que ambas poseen claves foráneas entre sí. Para resolver este inconveniente fue necesario controlar el orden de

inserción de los registros y adaptar el proceso de carga respetando las relaciones entre ambas tablas, el cual finalmente fue : actor, categoría, idioma, película, pelicula_actor, pelicula_categoria, tiend, empleado,cliente, inventario, alquiler y pago.
Otra dificultad importante fue la diferencia de sintaxis entre MySQL y SQL Server. También fue necesario aprender a dividir los archivos de gran tamaño utilizando bloques separados por GO

Justificación del método utilizado.
Se decidió realizar la migración mediante la adaptación manual de los scripts originales respetando los modelos de tablas de la actividad 1. Este método permitió comprender las diferencias existentes entre ambos motores de bases de datos, adaptar correctamente la sintaxis de cada sentencia, mantener la integridad referencial de la información y conservar los identificadores originales de todos los registros aunque fue algo lento.
De todas maneras, me permitió verificar cada etapa del proceso y resolver individualmente los problemas que fueron apareciendo durante la migración.

Conclusión.

La migración de la base de datos permitió comprender las principales diferencias entre MySQL y SQL Server, tanto en la sintaxis como en la administración de transacciones, restricciones y triggers.

Durante el desarrollo se presentaron distintos inconvenientes que requirieron adaptar los scripts originales, controlar las relaciones entre tablas y modificar algunos objetos propios de MySQL para obtener un comportamiento equivalente en SQL Server.
Como resultado, todas las tablas fueron migradas correctamente y se verificó que la cantidad de registros coincidiera con la base de datos original, garantizando la integridad y consistencia de la información.

Anexo de Trazabilidad de IA

1. Bitácora de Prompts

Durante el desarrollo de la migración se utilizó una herramienta de inteligencia artificial como apoyo para comprender diferencias entre MySQL y SQL Server, adaptar scripts y resolver errores surgidos durante la carga de datos.

Prompts utilizados

Prompt 1
Adaptar el script de creación de la base de datos Sakila desde MySQL a SQL Server, traduciendo tablas y columnas al español.
Resultado: Se obtuvo una primera versión del esquema que posteriormente fue revisada y ajustada manualmente para adaptar tipos de datos, restricciones, claves foráneas y nombres de columnas.

Prompt 2
Convertir los archivos INSERT de MySQL a SQL Server manteniendo los mismos datos.
Correcciones realizadas: eliminación de SET AUTOCOMMIT, eliminación de COMMIT, incorporación de IDENTITY_INSERT, traducción de tablas y columnas, adaptación del formato de fechas, división de archivos grandes en bloques de hasta 1000 registros separados por GO.

Prompt 3
Resolver errores producidos por claves foráneas durante la carga de datos.
Correcciones realizadas: ee identificó la dependencia circular entre las tablas empleado y tienda, se modificó el orden de carga de los datos, se realizaron pruebas hasta obtener una carga consistente.

Prompt 4
Explicar el funcionamiento de SET XACT_ABORT ON, TRY...CATCH y las transacciones en SQL Server.

Resultado: La IA explicó el funcionamiento de estas instrucciones y, luego de comprenderlas, fueron incorporadas a los scripts como una buena práctica para evitar cargas parciales en caso de error.

Prompt 5
Adaptar los triggers de MySQL para SQL Server.
Correcciones realizadas: Se analizó cada trigger manualmente.
Se determinó que varios no eran necesarios durante la migración porque los datos ya incluían los valores correspondientes.
En el caso de la tabla pago, la funcionalidad fue reemplazada por un DEFAULT GETDATE().

2. Correcciones manuales realizadas

Durante el proceso se realizaron distintas modificaciones sobre el código generado por la IA.
Entre ellas: cambio de tipos de datos para adecuarlos a SQL Server, ajuste de nombres de tablas y columnas al modelo traducido.
Corrección de claves foráneas, división de cargas masivas utilizando GO, incorporación de IDENTITY_INSERT, verificación de la cantidad de registros mediante consultas COUNT(*), resolución manual de errores de carga producidos por dependencias entre tablas.

3. Crítica del código generado por IA

La inteligencia artificial me permitió acelerar muchísimo el proceso de adaptación entre MySQL y SQL Server, especialmente en tareas repetitivas como la conversión de la sintaxis de los scripts de inserción y la explicación de conceptos propios de SQL Server.
Sin embargo, el código generado requirió revisión y correcciones manuales. Fue necesario ajustar nombres de tablas y columnas para que coincidieran con el esquema traducido anteriormente, modificar restricciones, resolver problemas de claves foráneas, dividir los archivos de gran tamaño respetando las limitaciones de SQL Server y verificar que la cantidad de registros migrados coincidiera con la base de datos original.

4. Comparación entre una consulta generada por IA y una versión ajustada manualmente

Código generado inicialmente

```
INSERT INTO payment VALUES
(1,1,1,76,2.99,'2005-05-25 11:30:37','2006-02-15 22:12:30');
```

Código ajustado manualmente para SQL Server

```
SET IDENTITY_INSERT pago ON;
GO

INSERT INTO pago
(
   pago_id,
   cliente_id,
   empleado_id,
   alquiler_id,
   monto,
   fecha_pago,
   ultima_actualizacion
)
VALUES
(1,1,1,76,2.99,'2005-05-25 11:30:37','2006-02-15 22:12:30');
GO
SET IDENTITY_INSERT pago OFF;
```

Justificación de los cambios

La versión generada inicialmente no contemplaba las diferencias entre MySQL y SQL Server. Fue necesario indicar explícitamente las columnas, utilizar IDENTITY_INSERT, traducir los nombres al español y dividir la carga en bloques separados por GO para cumplir con las restricciones de SQL Server.

Consulta propuesta inicialmente por la IA

```
SELECT TOP 10
   customer.customer_id,
```

```
    CONCAT(customer.first_name,' ',customer.last_name) AS
customer_name,
    COUNT(rental.rental_id) AS rentals,
    SUM(payment.amount) AS total_paid
FROM customer
JOIN rental
    ON customer.customer_id = rental.customer_id
JOIN payment
    ON rental.rental_id = payment.rental_id
GROUP BY
    customer.customer_id,
    customer.first_name,
    customer.last_name
ORDER BY total_paid DESC;
```

Consulta adaptada al proyecto

```
SELECT TOP 10
    c.cliente_id,
    CONCAT(c.nombre,' ',c.apellido) AS nombre_completo,
    COUNT(DISTINCT a.alquiler_id) AS cantidad_peliculas_alquiladas,
    SUM(p.monto) AS total_pagado
FROM cliente c
INNER JOIN alquiler a
    ON c.cliente_id = a.cliente_id
INNER JOIN pago p
    ON a.alquiler_id = p.alquiler_id
GROUP BY
    c.cliente_id,
    c.nombre,
    c.apellido
ORDER BY total_pagado DESC;
```

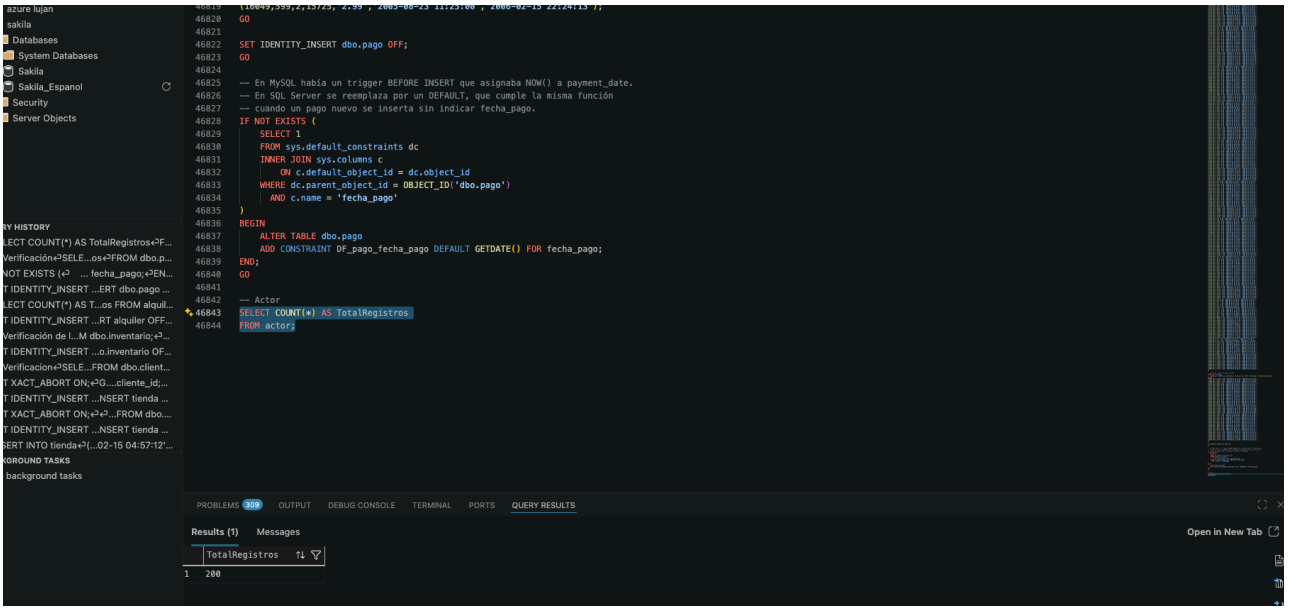
Análisis crítico

La consulta generada inicialmente utilizaba los nombres originales de la base de datos Sakila en MySQL (customer, rental, payment, first_name, last_name, etc.). Para poder utilizarla en el proyecto fue necesario adaptarla al esquema creado en SQL Server.

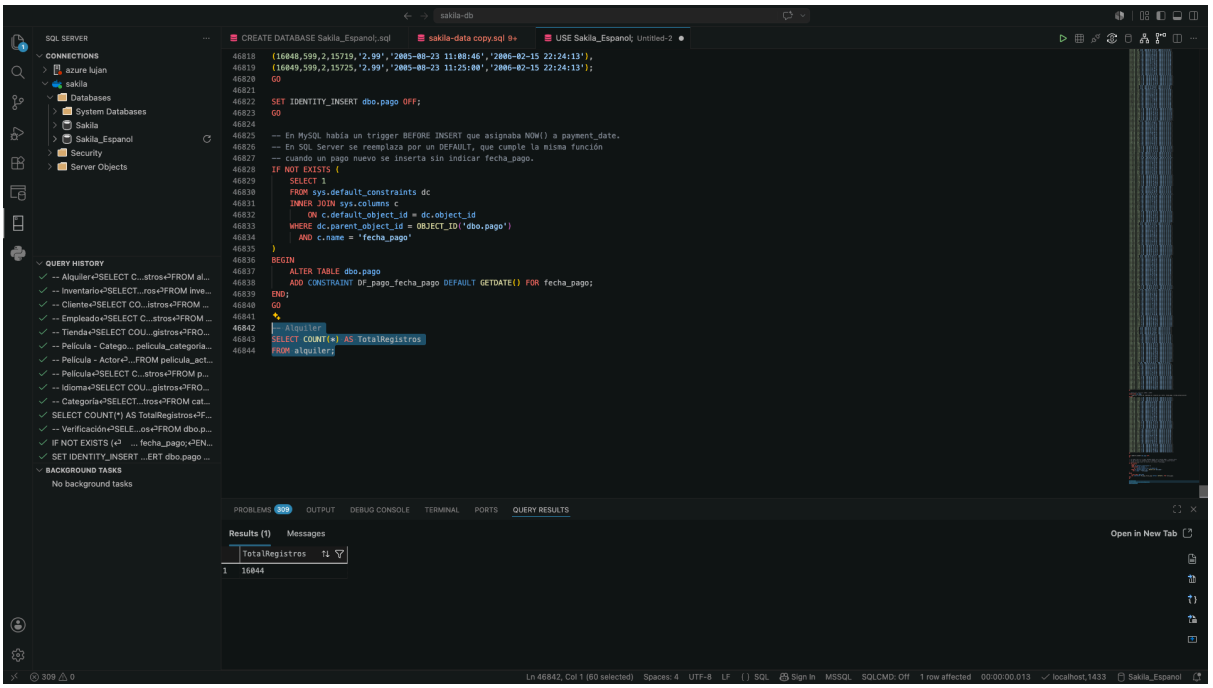
Se agregó COUNT(DISTINCT alquiler_id) para asegurar que cada alquiler se contabilizara una única vez.

Capturas.

Actor:

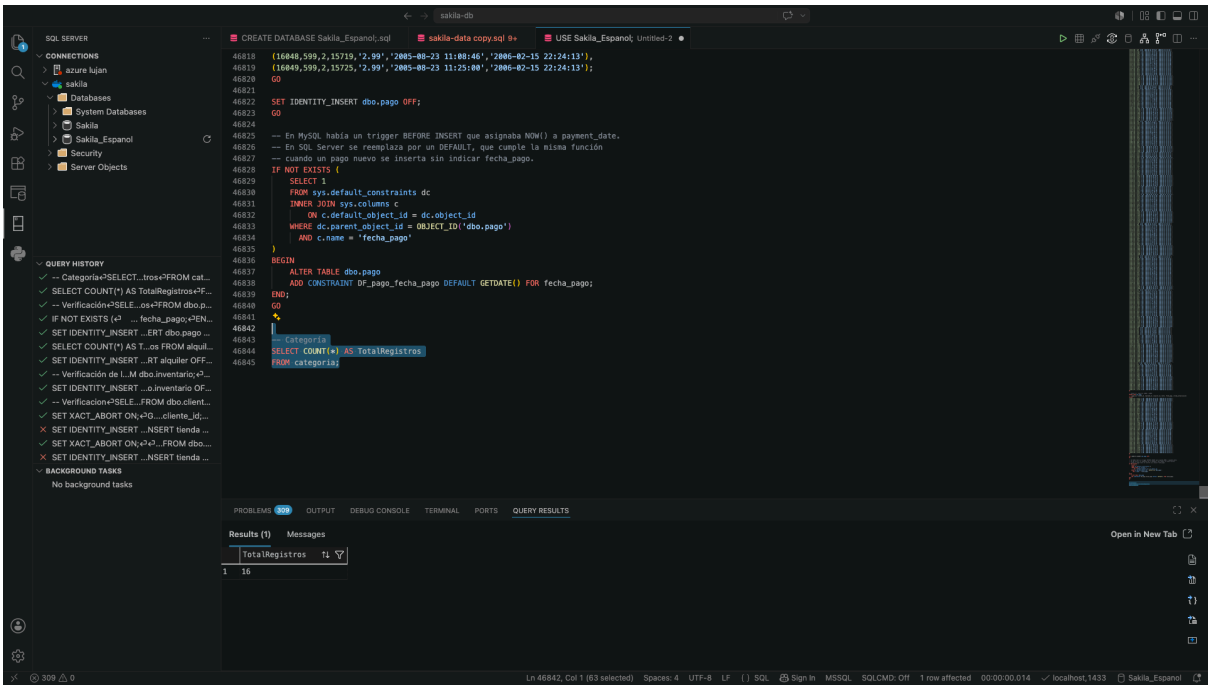


Alquiler:

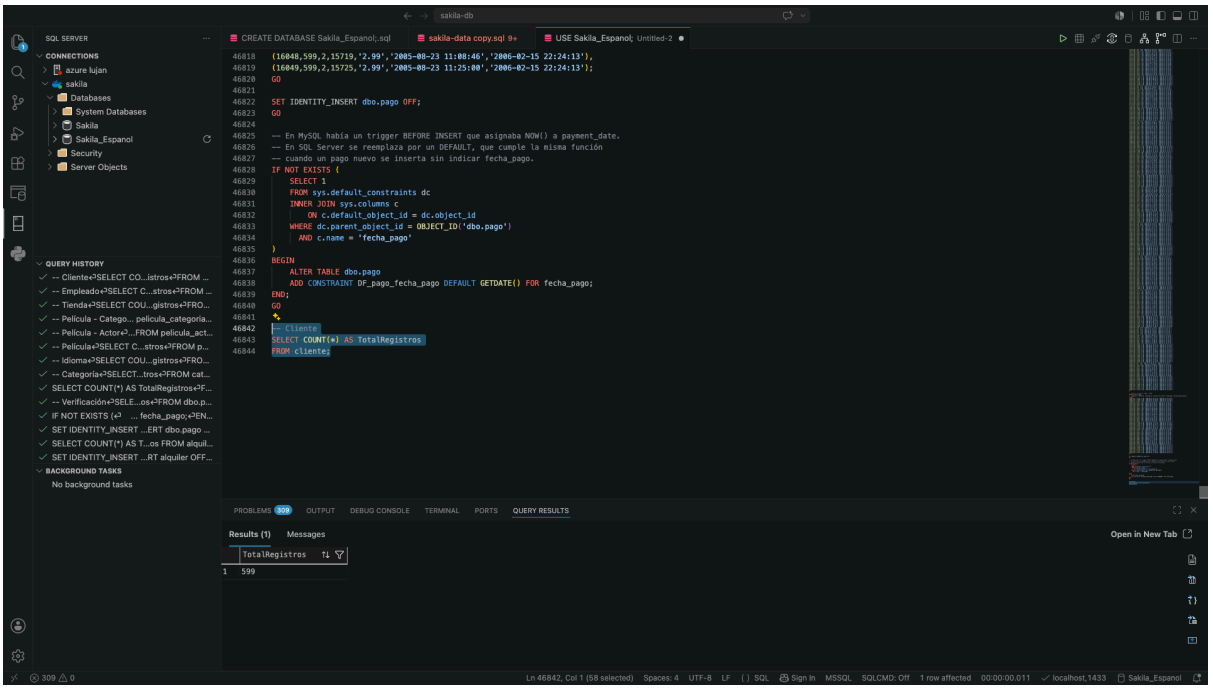


Lening, Dalma.

Categoría:

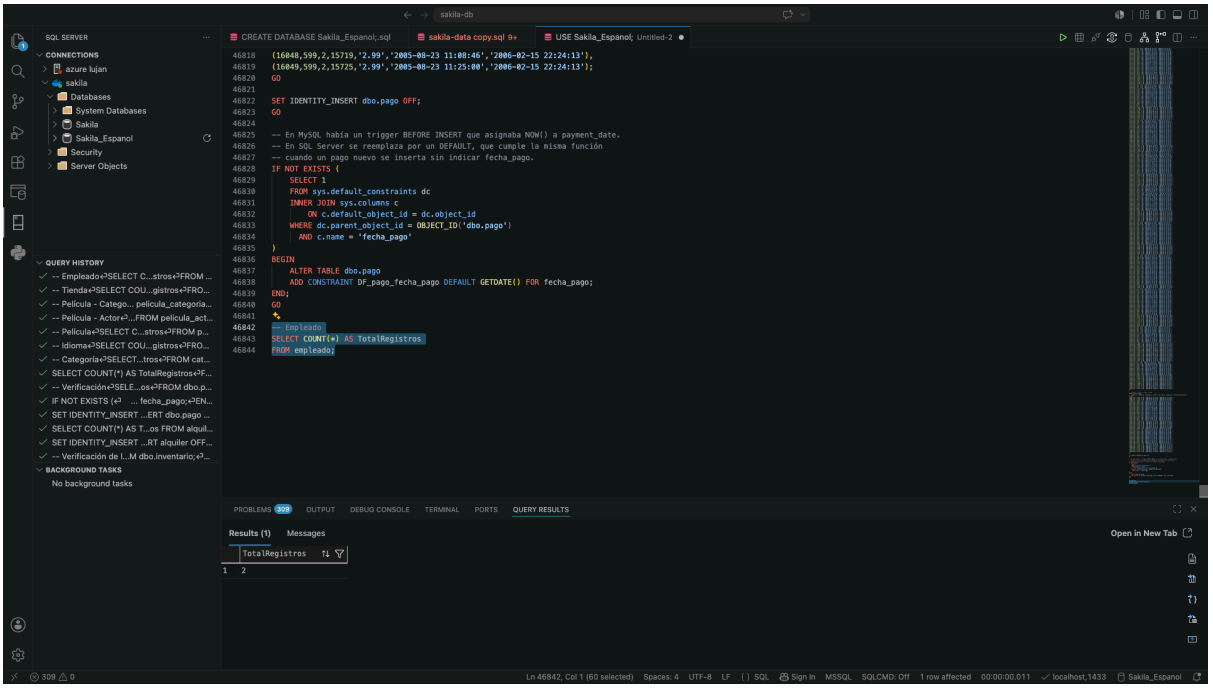


Cliente:

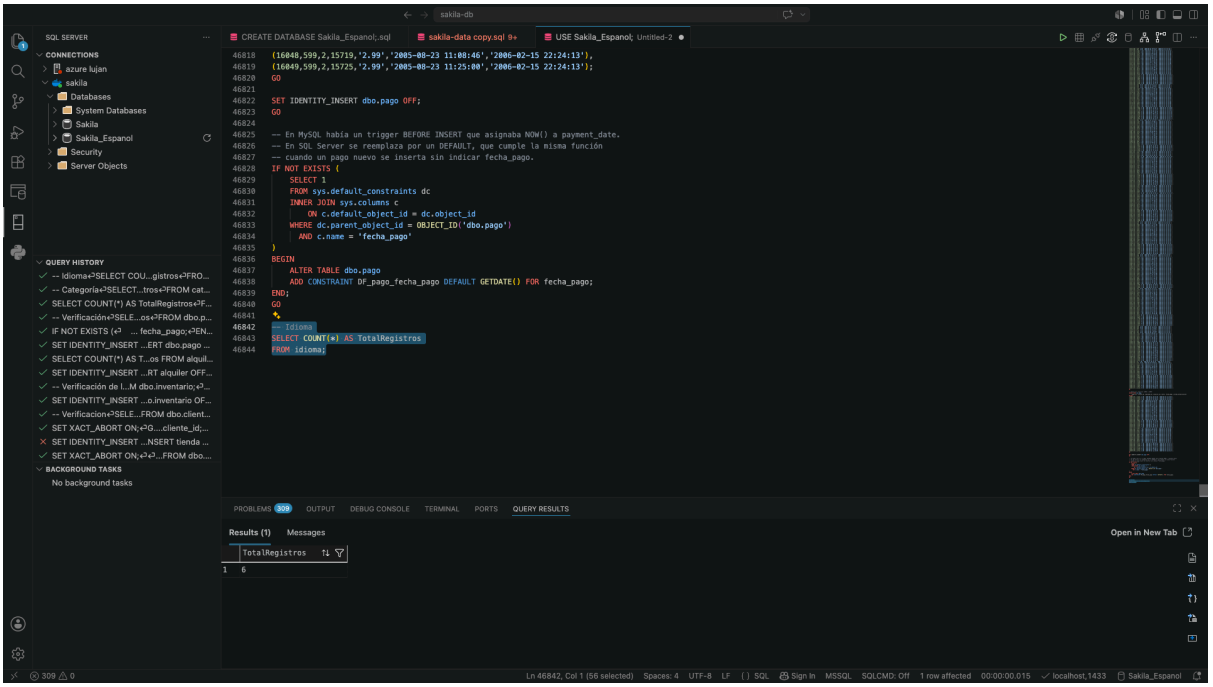


Lening, Dalma.

Empleado:

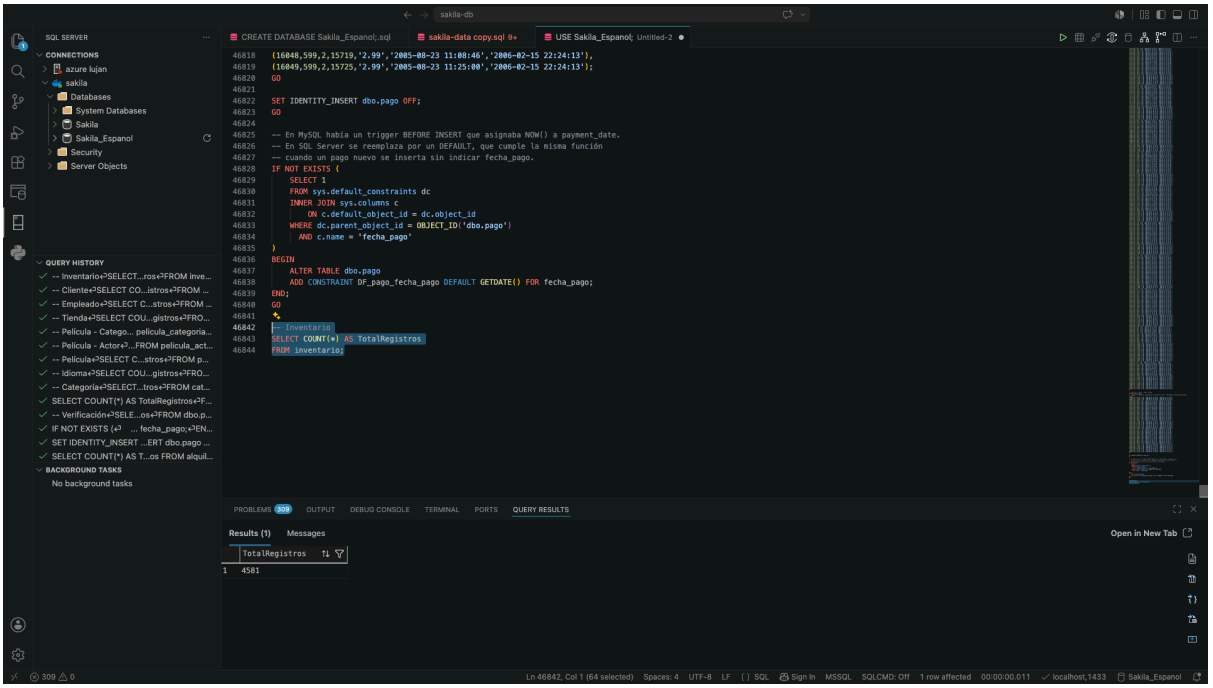


Idioma:

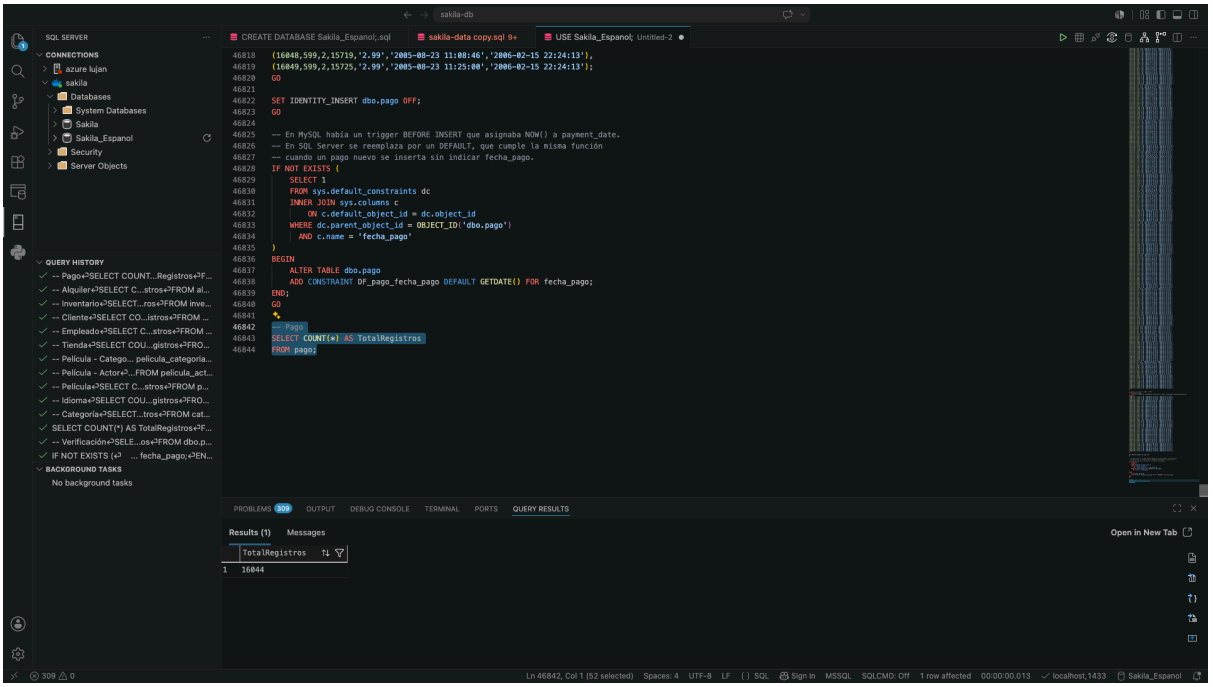


Lening, Dalma.

Inventario:

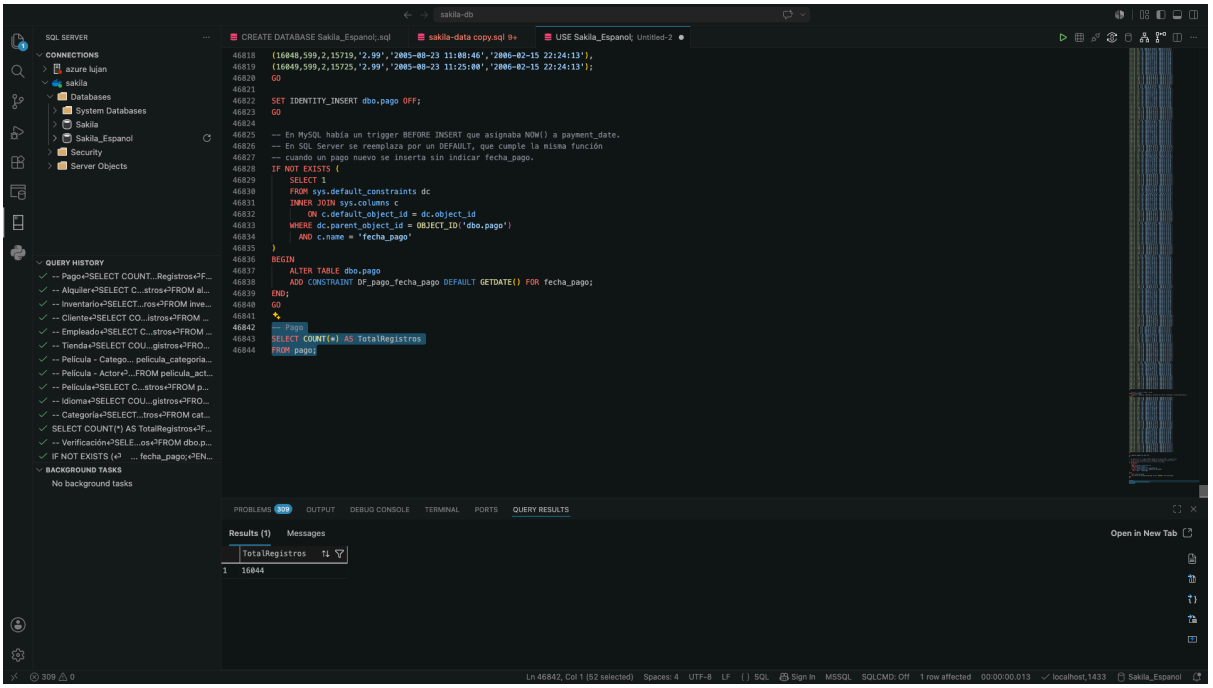


Pago:

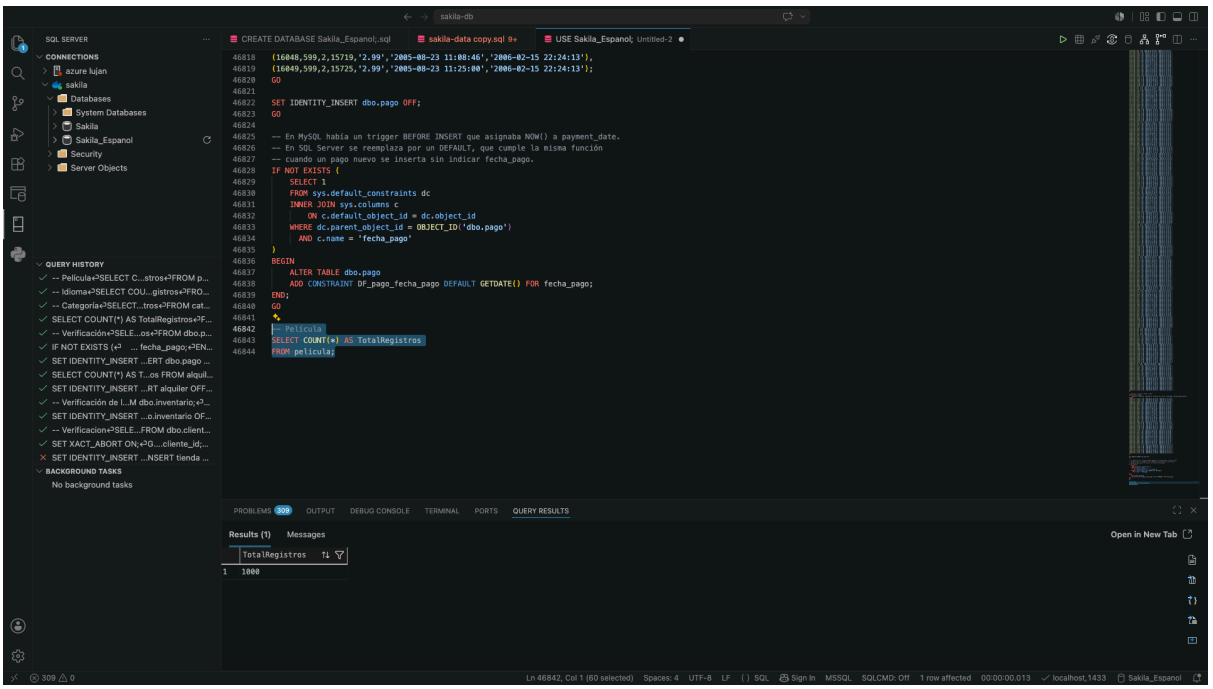


Lening, Dalma.

Pago:

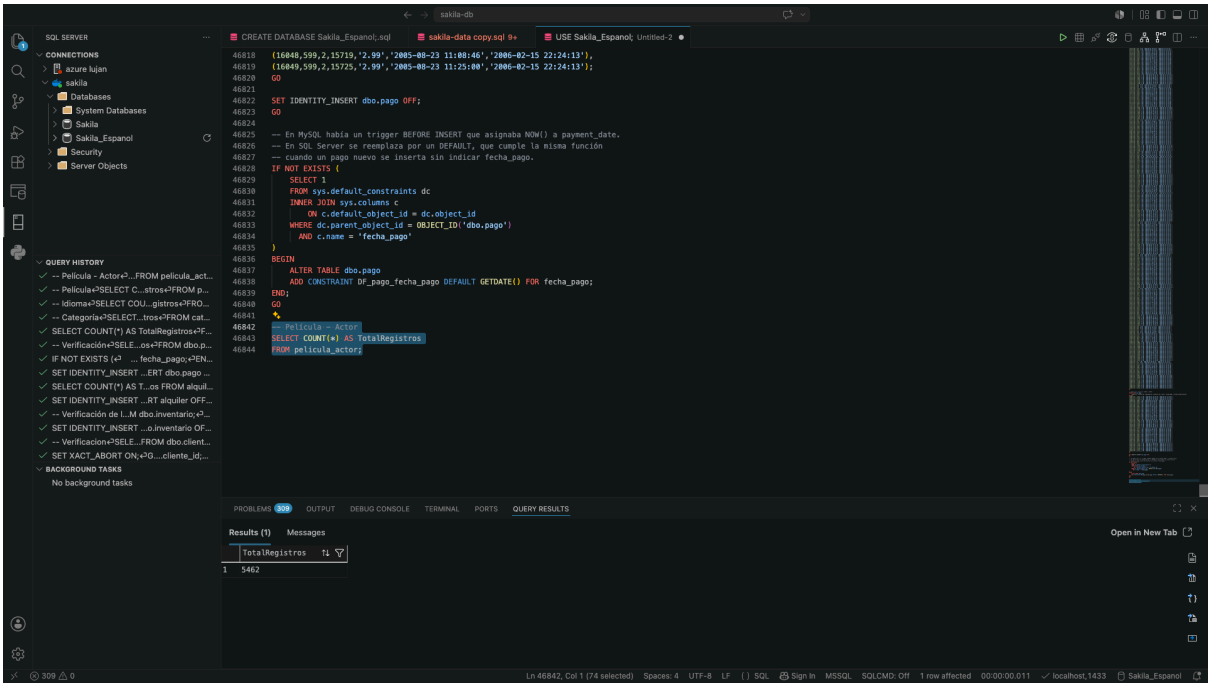


Película:

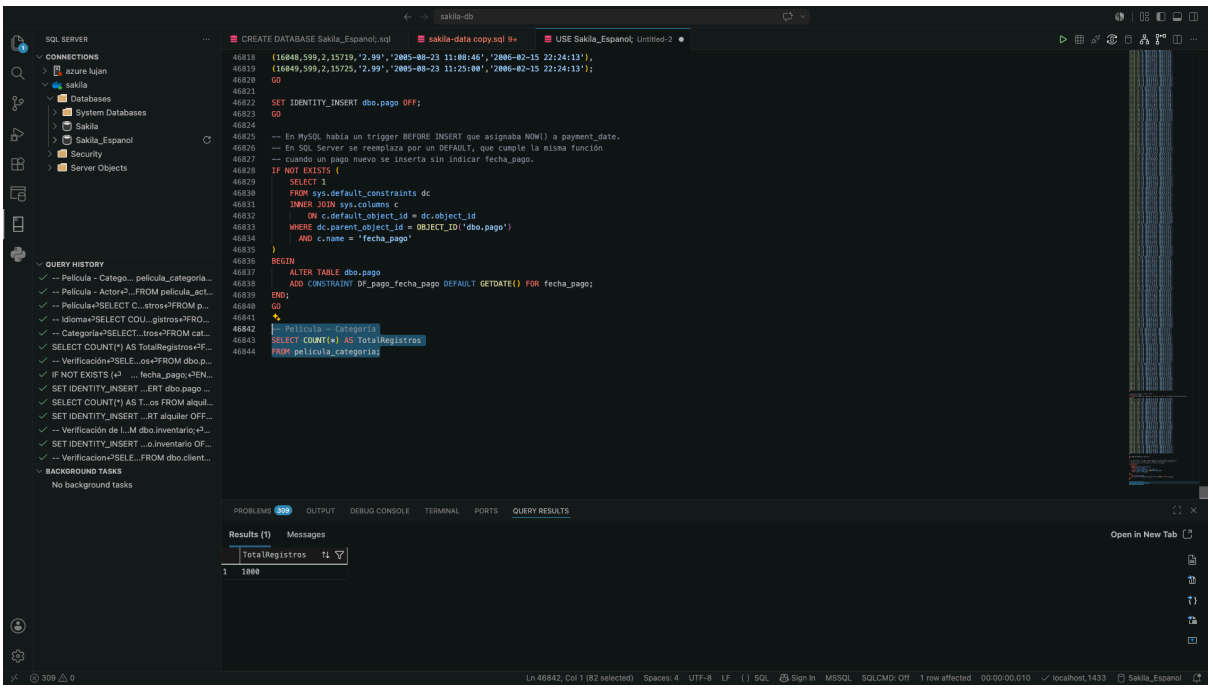


Película_actor:

Lening, Dalma.

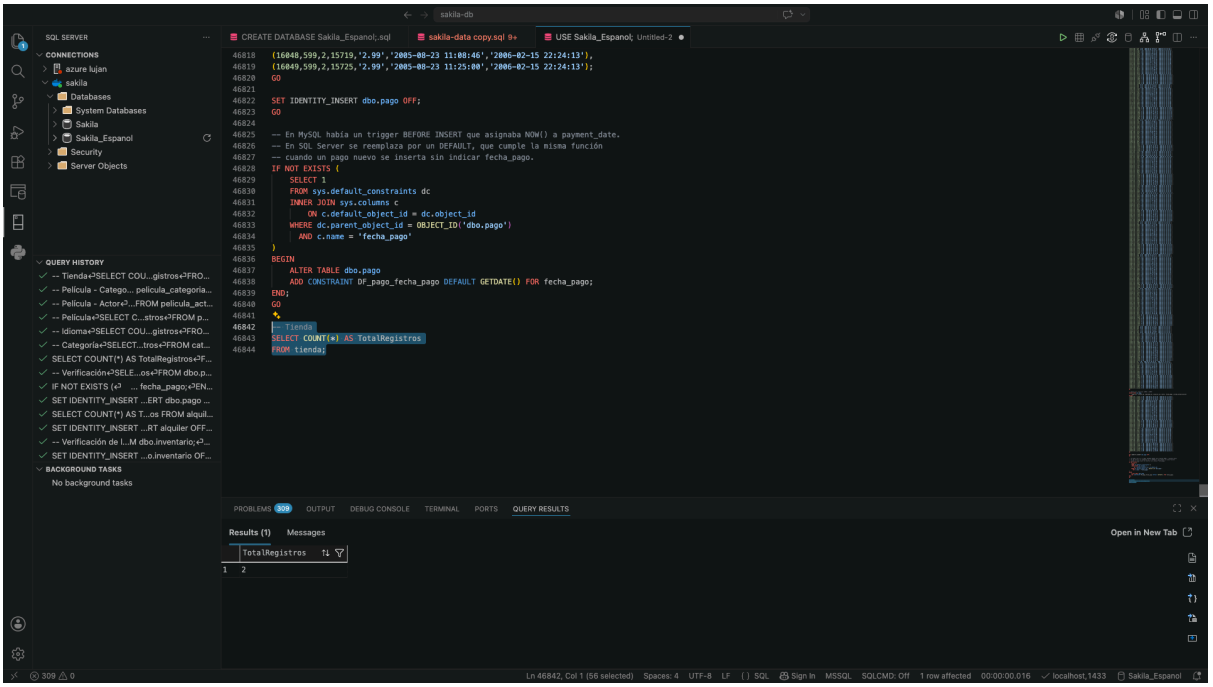


Película_categoría:



Tienda:

Lening, Dalma.



Lening, Dalma.